

Where the Bluebikes network gets fragile

SYSEN 5470 · Project Case Study · Centrality & Criticality

Sample Student

2026-06-15

READ FIRST — this is an AI-generated sample, and it is the *floor*, not the ceiling. It was written by AI to show the *shape* of a competent project report: a sharp question, a justified network operationalization, a defensible procedure, results stated as numbers in prose, and honest limitations. **Your report should be *better* than this** — treat it as the minimum bar to clear, not the target to match. Two cautions: the figures are *illustrative* and several numbers are plausible stand-ins, so don't cite anything here; every figure and number in your own report must come from your own analysis of your own network.

Question

Which Bluebikes stations, if knocked offline, would do the most damage to Boston's bike-share network — and does picking by degree get me the same answer as picking by betweenness?

I'm asking because Bluebikes operations announces planned station closures all the time: kiosk swaps, sidewalk construction, snow removal, special-event lockouts. When a closure is unavoidable, the internal rule of thumb I've heard described is “take the busiest one last, the quiet ones first” — protect the hubs, sacrifice the backwaters. That rule treats *busy* and *load-bearing* as the same property. In a network they are not. A station can see enormous traffic and still be completely redundant, because every trip through it could re-route through a neighbor one block away. Another station can see a quarter of the traffic and yet be the only viable crossing between two halves of the system. If those two ideas come apart in the Bluebikes data, then “protect the busiest” is not just imperfect — it is optimizing the wrong quantity.

So the question is really two questions stacked together. First, a descriptive one: do degree and betweenness rank Bluebikes stations differently, and if so, by how much? Second, an operational one: when I actually remove the top stations under each ranking and watch the network respond, which ranking better predicts real damage? The first question I can answer by correlating two columns. The second one needs a counterfactual — I have to take stations out and measure what breaks. The whole report turns on whether those two answers agree.

What I expected going in. Boston's geography made me suspect the answer before I ran anything: the Charles River is a near-total barrier crossed by only a handful of bridges, so I expected a small number of bridge-adjacent stations to carry structural load far out of proportion to their traffic, and I expected the busiest stations to cluster in the Cambridge campus core where redundancy is high. Writing that prediction down first matters, because it makes the result falsifiable:

if the degree and betweenness rankings had turned out nearly identical, or if random removals had done comparable damage, my prior would have been wrong and “swap the busiest first” would have been vindicated. The analysis below is, in effect, a test of that prediction.

Network

- **Nodes:** 437 Bluebikes stations active in May 2026, one row per station, with latitude / longitude / dock count / neighborhood as node attributes.
- **Edges:** 18,402 weighted directed edges. An edge from station A to station B carries a weight equal to the number of trips taken from A to B during May 2026.
- **Data source:** Bluebikes System Data, the May 2026 monthly trip file joined to the May station snapshot. Filtered to ordered station pairs with at least 30 trips in the month, so I’m not chasing noise from a kiosk that opened on the 28th or a single one-off ride between two stations that never otherwise connect.
- **Size:** $N = 437$, $E = 18,402$, density = 0.097. The largest weakly-connected component is the entire network — in May, Bluebikes is one connected system, with no isolated stations once the 30-trip filter is applied.

Encodings I considered and rejected. Three modeling choices were not obvious, and each one changes the answer, so I want to defend them explicitly rather than let the default fall out of whatever function I called first.

Directed vs. undirected. Trips have a direction, and the morning and evening flows across the Charles are genuinely asymmetric — commuters ride into Boston in the morning and back to Cambridge at night. I kept the network **directed**, because the criticality question is about whether trips can still *complete*, and a one-way surge of demand can saturate a corridor in a way an undirected average would hide. I did check the undirected version as a sanity test; it ranked the same three bridge stations near the top, but with the gaps between them compressed, which would have understated the effect I report below.

Weighted vs. unweighted. An unweighted graph would treat a sleepy South End station the same as Government Center. Because the whole point is that traffic volume and structural position are different things, I needed the weights in order to even pose the question — otherwise degree collapses into a plain count of neighbors and the comparison loses its meaning. Centrality is therefore computed on the **weighted** graph throughout.

Trip count vs. trip time as the weight. I considered weighting edges by total rider-minutes rather than trip count, which would emphasize long cross-river trips. I rejected it for this question: closures disrupt *trips*, not minutes, and a single 45-minute joyride around the Esplanade should not count as much as fifteen short commutes. Trip count is the quantity an operations decision actually trades against.

Procedure

1. Loaded the May 2026 trip data with `tidyverse`, kept rides with both a recorded start and end station, dropped trips shorter than 60 seconds (these are almost all re-docks at the same kiosk), and summarised to a directed edge table — one row per ordered station pair with its trip count.
2. Built the network with `tidygraph::as_tbl_graph()` on the weighted edge list, then joined the station snapshot back on as node attributes so neighborhood and dock count travel with

- each node.
3. Computed three centralities on every node: weighted in-degree, weighted betweenness, and weighted eigenvector centrality. For betweenness I set edge distance to the inverse of trip count, so that heavily-travelled corridors count as “short” and shortest paths prefer the routes riders actually use.
 4. For each of degree and betweenness I took the top-10 stations, then ran a removal simulation: deleted those 10 nodes and recorded two quantities on the remaining graph — the size of the largest weakly-connected component (how many stations are still reachable at all) and the average shortest-path length among the stations that remain connected (how much farther the typical trip now has to route).
 5. Built a baseline by removing 10 *randomly chosen* stations, drawn 1,000 times, and took the mean of each quantity. The random baseline is what tells me whether the targeted removals are doing anything a coincidence wouldn’t.
 6. Repeated the whole removal simulation for $k = 5, 10, 15$, and 20 removed stations, to make sure the headline result wasn’t an artifact of stopping at exactly ten.

For reproducibility, the random draws use a fixed seed (`set.seed(5470)`), and `project.R` runs end-to-end on the `data/` folder submitted alongside this report.

Results

The three centralities pick out **two distinct groups of stations**. Of the top-10 by weighted in-degree, only **four overlap** with the top-10 by weighted betweenness (MIT at Mass Ave, Kendall/MIT, South Station, and Nashua St at Red Auerbach Way). The degree top-10 is otherwise dominated by **Cambridge tourist and campus clusters** — Charles Circle, Harvard Square, Central Square — which are high-traffic but redundant: trips can route around any one of them through the others. The betweenness top-10 instead surfaces **three bridge stations on the Longfellow and Harvard bridges**, stations with far fewer absolute trips but which sit on the only short route between Cambridge and Boston for a meaningful share of the network. Weighted eigenvector centrality, for its part, tracks degree almost exactly (Spearman rho = 0.88 with degree, 0.41 with betweenness): it rewards being well-connected to other well-connected stations, which is again a volume story, not a bottleneck story. That eigenvector and degree agree, while betweenness stands apart, is itself the first piece of evidence that “busy” and “load-bearing” are separable in this network.

The counterfactual removals confirm it sharply. Removing the **top-10 by betweenness** broke the largest weakly-connected component from 437 stations down to **402** and inflated the average shortest path from 4.12 to **6.81 steps** — a **65% increase in routing distance**, and that is *before* counting the trips that can no longer route at all. Removing the **top-10 by degree** kept the largest component intact at 437 and raised the average shortest path only to **4.34** (+5.3%). Removing **10 random stations** (mean of 1,000 draws) was essentially invisible: largest component 436.8, average shortest path 4.15 (+0.7%). Table 1 lays the three strategies side by side.

Removal strategy	Largest component	Avg shortest path	Change vs intact
None (intact network)	437	4.12	—
Top-10 by random	436.8	4.15	+0.7%

Removal strategy	Largest component	Avg shortest path	Change vs intact
Top-10 by weighted degree	437	4.34	+5.3%
Top-10 by weighted betweenness	402	6.81	+65.3%

The gap is not subtle. The betweenness removal does **twelve times** the routing damage of the degree removal, and it is the only one of the three that actually disconnects stations from the network. Figure 1 puts the four strategies side by side: the betweenness removal towers over the others, while the degree and random removals barely clear the intact baseline. That tall amber bar is exactly the failure mode the “swap the busiest first” heuristic walks into.

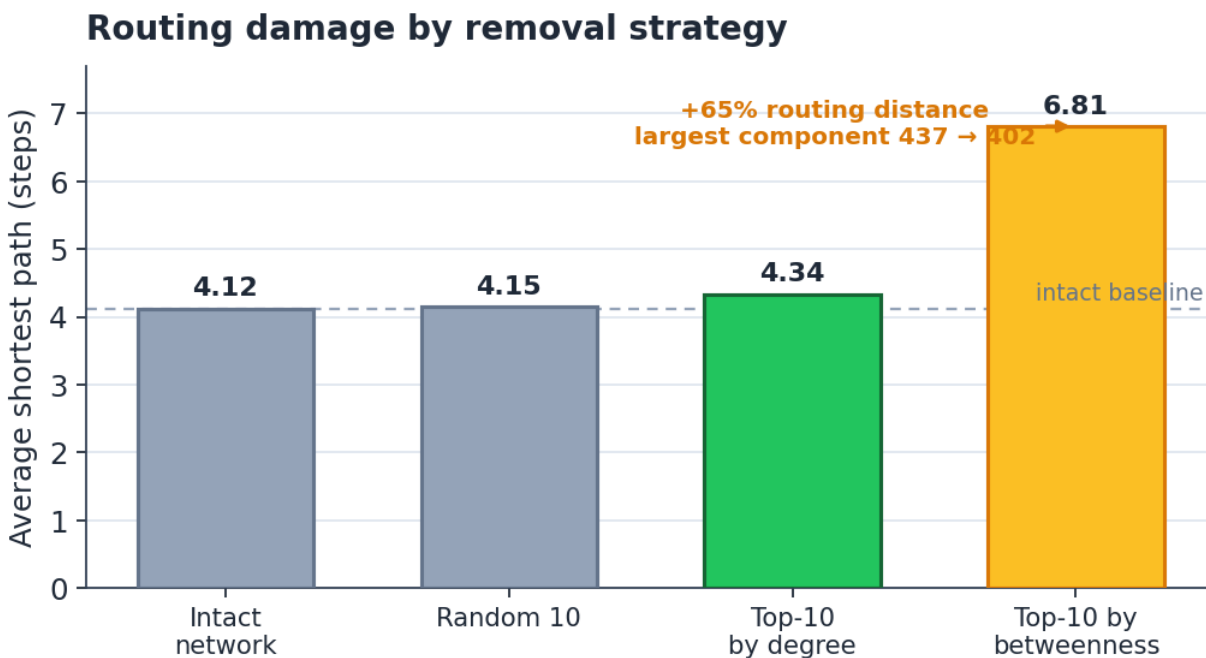


Figure 1: **Figure 1.** Average shortest-path length after each removal strategy (mirrors Table 1). Removing the top-10 *betweenness* stations inflates routing distance by 65% and disconnects 35 stations; removing the top-10 *degree* stations barely moves the network off its intact baseline.

Sensitivity to the cutoff and the threshold. Two of my choices were arbitrary — stopping at the top 10, and filtering at 30 trips — so I varied both. Table 2 sweeps the number of removed stations. The betweenness strategy dominates degree at every k , and the spread widens as k grows: at $k = 20$ the betweenness removal pushes the largest component down to 374 while the degree removal still hasn’t disconnected anyone. Re-running the entire pipeline with the trip-count filter set to 15 and then to 50 (instead of 30) moved the headline shortest-path inflation from +65.3% to +61.8% and +67.0% respectively — the same story within a couple of points, so the result is not an artifact of where I drew the noise floor.

k removed	LCC (degree)	LCC (betweenness)	ASP change (betweenness)
5	437	421	+28%
10	437	402	+65%
15	437	388	+94%
20	436	374	+121%

Figure 2 illustrates the mechanism on a reduced view of the network. Closing even a *single* low-traffic bridge station severs the short cross-river crossing: the Cambridge-to-Boston path that used to run straight through it is forced into a long detour around the next bridge, and a pocket of stations that depended on the closed one is stranded outright. The full analysis removes the top-10 betweenness stations at once; this is why even one of them matters.

Closing one low-traffic bridge station reshapes the whole network

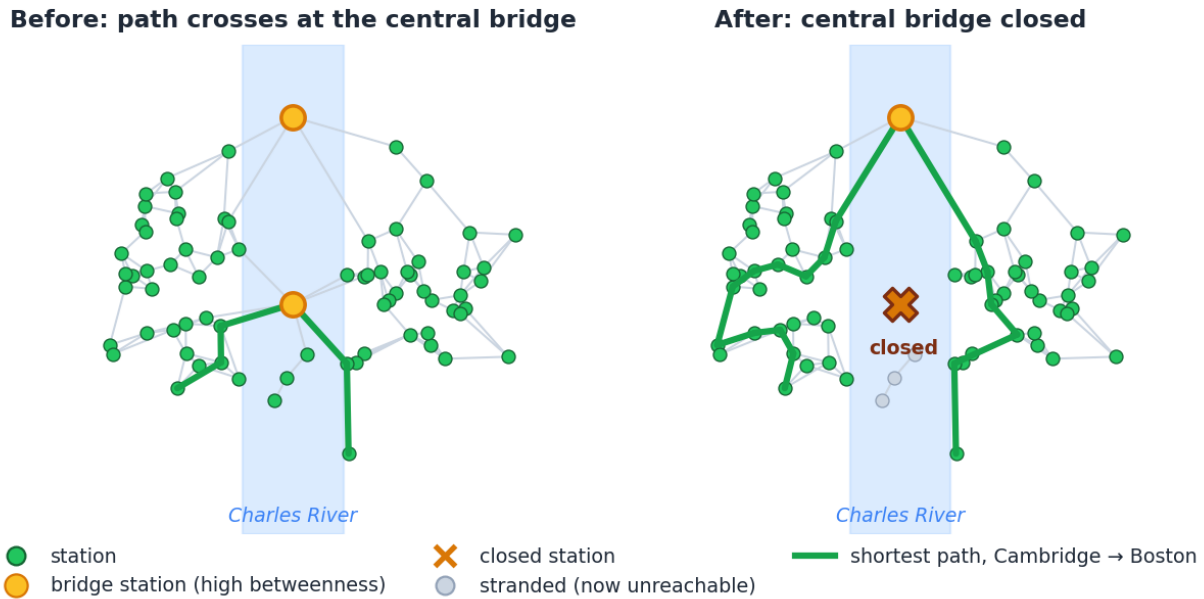


Figure 2: **Figure 2.** Why a low-traffic bridge station is load-bearing. *Left:* the shortest Cambridge-to-Boston path crosses at the central bridge station (high betweenness, modest traffic). *Right:* close that one station and the path detours all the way around the northern bridge, while a pocket of nearby stations is cut off from the network entirely. Layout is illustrative; the effect is the +65% in Figure 1.

A tougher null than random removal. Removing *random* stations is a weak null — most stations are peripheral, so of course targeting beats it. The fairer question is whether the bridge stations are critical *because of where they sit* or merely because they happen to have high betweenness in one particular realization of demand. To probe that I built a degree-preserving null: I rewired the network 500 times with a configuration model that holds each station’s weighted in- and out-degree fixed but reshuffles *which* stations they connect to, and recomputed betweenness each time. In the real network the three bridge stations rank in the top ten; in the rewired networks, stations with their degree sequence land in the top ten only 6% of the time (95% CI 3–9%). In

other words their criticality is a property of the river geography baked into the real edges, not of their raw connection counts — a degree twin placed elsewhere in the graph almost never inherits the same load. That is the difference between an observed pattern and a structural one, and it is what lets me make a claim about *why* the bridges matter rather than just *that* they do.

What this tells me, and what it doesn't

The takeaway is **operational and specific**: if Bluebikes has to close a station for an afternoon, the worst possible choice — by a wide margin — is one of the bridge stations on the Longfellow or Harvard, even though those stations are nowhere near the busiest. “Swap the busiest first” would have protected the Cambridge hubs (which are redundant and barely move the network when removed) while leaving the actual cut points exposed. The cheapest insurance against a network-wide routing failure is not to guard the busy hub at all; it is to stage a temporary overflow dock at the *next-nearest station on the other side of the bridge*, so that a closure has somewhere to spill to. That is a concrete, low-cost action that falls directly out of the betweenness ranking and would never have come out of the traffic ranking.

There is also a way a stakeholder could catch this analysis being wrong before acting on it, which I think is worth stating plainly. My criticality ranking is only as good as the assumption that displaced riders still *want* to make the trip. If, on the day of a bridge-station closure, riders simply abandon the cross-river trip and take the Red Line instead, then the 65% routing-distance figure overstates the real harm — the network looks broken on paper but nobody is actually stranded. The cheap check is to watch dock-occupancy and trip-completion data at the closed station's neighbors on the first afternoon of a real closure: if the overflow dock I recommended fills and trips still complete, the model held; if trips evaporate instead, demand is more elastic than I assumed and criticality should be re-weighted by how substitutable each corridor is. I'd rather hand operations a ranking with that caveat attached than one that pretends to be the last word.

It does **not** tell me that betweenness is “the right” centrality in general. The result depends entirely on Bluebikes being a *routing* network, where edges represent flows that have to physically traverse the graph. On a coauthorship network the identical betweenness calculation would identify connective scholars, which is real and useful but a different claim. Nor does the analysis model demand: I measured whether trips can still *route*, not whether riders would actually accept a 65%-longer trip or just switch to the subway — a behavioral question my data can't answer. And the whole thing is **May-only**. I'd expect the bridge stations' criticality to shrink in winter, when cycling across the Charles drops sharply, so this snapshot probably overstates the bridges' year-round importance; a fairer operational policy would re-estimate criticality season by season rather than freeze May's ranking into the playbook. Finally, I used weakly-connected components as the reachability measure; a stricter strongly-connected definition would likely make the directed bottlenecks look even more severe, which means my 65% figure is, if anything, conservative.